

下一步

使用思考模型

📄 复制页面



推荐使用 `kimi-k2.6` 或 `kimi-k2.5` 的思考能力：默认启用，可通过 `"thinking": {"type": "disabled"}` 禁用思考能力。

如果您使用 kimi api 进行基准测试，请参考这篇 [基准测试最佳实践](#)

基本示例

使用 Kimi K2.6 模型

你可以通过 `kimi-k2.6` 使用思考能力：

`curl` `python`

KIMI 开放平台

```
-H "Content-Type: application/json" \  
-H "Authorization: Bearer $MOONSHOT_API_KEY" \  
-d '{  
  "model": "kimi-k2.6",  
  "messages": [  
    {  
      "role": "system",  
      "content": "你是 Kimi。"  
    },  
    {  
      "role": "user",  
      "content": "请解释 1+1=2。"  
    }  
  ],  
  "temperature": 1.0  
'
```

使用 Kimi K2.6 模型启用思考能力

对于 `kimi-k2.6`，`kimi-k2.5` 模型，默认启用思考能力，无需在调用时手动指定：

`curl` `python`

KIMI 开放平台

```
-H "Content-Type: application/json" \  
-H "Authorization: Bearer $MOONSHOT_API_KEY" \  
-d '{  
  "model": "kimi-k2.6",  
  "messages": [  
    {  
      "role": "system",  
      "content": "你是 Kimi。"  
    },  
    {  
      "role": "user",  
      "content": "请解释 1+1=2。"  
    }  
  ]  
'
```

使用 Kimi K2.6 模型并禁用思考能力

请参阅 [k2.6 禁用思考能力示例](#)

输出思考内容

注意到，在使用 `kimi-k2.6` 或 `kimi-k2.5`（启用思考能力时）模型时，我们的 API 响应中使用了 `reasoning_content` 字段作为模型思考内容的载体，对于 `reasoning_content` 字段：

- openai SDK 中的 `ChoiceDelta` 和 `ChatCompletionMessage` 类型并不提供 `reasoning_content` 字段，因此无法直接通过 `.reasoning_content` 的方式访问该字段，仅支持通过 `hasattr(obj, "reasoning_content")` 来判断是否存在字段，如果存在，则使用 `getattr(obj, "reasoning_content")` 获取字段值
- 如果你使用其他框架或自行通过 HTTP 接口对接，可以直接获取与 `content` 字段同级的 `reasoning_content` 字段
- 在流式输出（`stream=True`）的场合，`reasoning_content` 字段一定会先于 `content` 字段出现，你可以在业务代码中通过判断是否出现 `content` 字段来识别思考内容（或称推理过

- `reasoning_content` 中包含的 Tokens 也受 `max_tokens` 参数控制，**KIMI 开放平台** `reasoning_content` 的 Tokens 数加上 `content` 的 Tokens 数应小于等于 `max_tokens`。

多步工具调用

`kimi-k2.6` 和 `kimi-k2.5`（启用思考能力时）都支持通过深度地推理进行多步工具调用，进而完成非常复杂的任务。

使用须知

为确保最佳效果，使用 `kimi-k2.6` 或 `kimi-k2.5`（通过 `thinking` 参数启用思考能力）时，请务必按以下方式配置调用：

- 输入应当包括上下文中所有的思考内容(`reasoning_content`字段)，模型会根据实际情况选择把必要的思考内容送到模型进行推理。
- 设置 `max_tokens>=16000` 以避免无法输出完整的 `reasoning_content` 和 `content`。
- 设置 `temperature=1.0`，以获得最佳性能。其中 `kimi-k2.6`，`kimi-k2.5` 模型固定使用 `temperature=1.0`。
- 使用流式输出（`stream=True`）：思考模型的输出内容包含了 `reasoning_content`，相比普通模型其输出内容更多，启用流式输出能获得更好的用户体验，同时一定程度避免网络超时问题。

完整示例

下面的示例展示了一个“今日新闻报告生成”的场景，模型会依次调用 `date`（获取日期）和 `web_search`（搜索今日新闻）等官方工具，并在这个过程中展现深度思考过程。

KIMI 开放平台
+ 代码

```
import json
import openai
```



```
class FormulaChatClient:
    def __init__(self, base_url: str, api_key: str):
        """初始化 Formula 客户端"""
        self.base_url = base_url
        self.api_key = api_key
        self.openai = openai.Client(
            base_url=base_url,
            api_key=api_key,
        )
        self.httpx = httpx.Client(
            base_url=base_url,
            headers={"Authorization": f"Bearer {api_key}"},
            timeout=30.0,
        )
        # 使用 kimi-k2.6 模型, thinking 将默认启用
        self.model = "kimi-k2.6"

    def get_tools(self, formula_uri: str):
        """从 Formula API 获取工具定义"""
        response = self.httpx.get(f"/formulas/{formula_uri}/tools")
        response.raise_for_status() # 检查 HTTP 状态码

        try:
            return response.json().get("tools", [])
        except json.JSONDecodeError as e:
            print(f"错误: 无法解析响应为 JSON (状态码: {response.status_code})")
            print(f"响应内容: {response.text[:500]}")
            raise

    def call_tool(self, formula_uri: str, function: str, args: dict):
        """调用官方工具"""
        response = self.httpx.post(
            f"/formulas/{formula_uri}/fibers",
            json={"name": function, "arguments": json.dumps(args)},
        )
```

KIMI

开放平台



```
fiber = response.json()
```

```
if fiber.get("status", "") == "succeeded":
```

```
> return fiber["context"].get("output") or fiber["context"].get("encrypt
```

```
if "error" in fiber:
```

```
    return f"Error: {fiber['error']}"
```

```
if "error" in fiber.get("context", {}):
```

```
    return f"Error: {fiber['context']['error']}"
```

```
return "Error: Unknown error"
```

```
def close(self):
```

```
    """关闭客户端连接"""
```

```
    self.httpx.close()
```

```
# 初始化客户端
```

```
base_url = os.getenv("MOONSHOT_BASE_URL", "https://api.moonshot.cn/v1")
```

```
api_key = os.getenv("MOONSHOT_API_KEY")
```

```
if not api_key:
```

```
    raise ValueError("MOONSHOT_API_KEY 环境变量未设置，请先设置 API 密钥")
```

```
print(f"Base URL: {base_url}")
```

```
print(f"API Key: {api_key[:10]}...{api_key[-10:]} if len(api_key) > 20 else api_key
```

```
client = FormulaChatClient(base_url, api_key)
```

```
# 定义要使用的官方工具 Formula URI
```

```
formula_uris = [
```

```
    "moonshot/date:latest",
```

```
    "moonshot/web-search:latest"
```

```
]
```

```
# 加载所有工具定义并建立映射
```

```
print("正在加载官方工具...")
```

```
all_tools = []
```

```
tool_to_uri = {} # function.name -> formula_uri 的映射
```

KIMI

开放平台



```
try:
    tools = client.get_tools(uri)
    for tool in tools:
        > func = tool.get("function")
        if func:
            func_name = func.get("name")
            if func_name:
                tool_to_uri[func_name] = uri
                all_tools.append(tool)
                print(f" 已加载工具: {func_name} from {uri}")
except Exception as e:
    print(f" 警告: 加载工具 {uri} 失败: {e}")
    continue

print(f"总共加载 {len(all_tools)} 个工具\n")

if not all_tools:
    raise ValueError("未能加载任何工具，请检查 API 密钥和网络连接")

# 初始化消息列表
messages = [
    {
        "role": "system",
        "content": "你是 Kimi，一个专业的新闻分析师。你擅长收集、分析和整理信息，生成高质量的",
    },
]

# 用户请求生成今日新闻报告
user_request = "请帮我生成一份今日新闻报告，包含重要的科技、经济和社会新闻。"
messages.append({
    "role": "user",
    "content": user_request
})

print(f"用户请求: {user_request}\n")

max_iterations = 10 # 防止无限循环
```

KIMI 开放平台



```
try:
    completion = client.openai.chat.completions.create(
        model=client.model,
        > messages=messages,
        max_tokens=1024 * 32,
        tools=all_tools,
        temperature=1.0,
    )
except openai.AuthenticationError as e:
    print(f"认证错误: {e}")
    print("请检查 API key 是否正确，以及 API key 是否有权限访问该端点")
    raise
except Exception as e:
    print(f"调用模型时发生错误: {e}")
    raise

# 获取响应
message = completion.choices[0].message

# 打印思考过程
if hasattr(message, "reasoning_content"):
    print(f"=====第 {iteration + 1} 轮思考开始=====")
    reasoning = getattr(message, "reasoning_content")
    if reasoning:
        print(reasoning[:500] + "... " if len(reasoning) > 500 else reasoning)
    print(f"=====第 {iteration + 1} 轮思考结束=====\\n")

# 添加 assistant 消息到上下文 (保留 reasoning_content)
messages.append(message)

# 如果模型没有调用工具，说明对话结束
if not message.tool_calls:
    print("=====最终回答=====")
    print(message.content)
    break

# 处理工具调用
print(f"模型决定调用 {len(message.tool_calls)} 个工具:\\n")
```


KIMI

开放平台



```
func_name = tool_call.function.name
args = json.loads(tool_call.function.arguments)

print(f"调用工具: {func_name}")
print(f"参数: {json.dumps(args, ensure_ascii=False, indent=2)}")

# 获取对应的 formula_uri
formula_uri = tool_to_uri.get(func_name)
if not formula_uri:
    print(f"错误: 找不到工具 {func_name} 对应的 Formula URI")
    continue

# 调用工具
result = client.call_tool(formula_uri, func_name, args)

# 打印结果 (截断过长内容)
if len(str(result)) > 200:
    print(f"工具结果: {str(result)[:200]}...\n")
else:
    print(f"工具结果: {result}\n")

# 添加工具结果到消息列表
tool_message = {
    "role": "tool",
    "tool_call_id": tool_call.id,
    "name": func_name,
    "content": result
}
messages.append(tool_message)

print("\n对话完成! ")

# 清理资源
client.close()
```

整个过程展现了 `kimi-k2.6` 或 `kimi-k2.5` (启用思考能力时) 模型如何通过深度思考来规划和执行复杂的多步骤任务, 每个步骤都有完整的推理过程 (`reasoning_content`), 并且思考内容会

什么是 Preserved Thinking

Preserved Thinking 指在多轮对话中，把历史轮次（previous turns）的 `reasoning_content` 一并透传给模型，让模型在本轮推理时能延续之前的思考脉络。

对于 `kimi-k2.6` 模型，可通过请求体中的 `thinking.keep` 参数控制是否保留历史思考：

取值

行为

`null` / 不传（默认）

忽略历史轮次的 `reasoning_content`，上下文更短、成本更低。

`"all"`

完整保留历史轮次的 `reasoning_content`，启用 Preserved Thinking。

❗ `thinking.keep` 只影响历史轮次的 `reasoning_content`，并不改变模型在当前轮次是否产生/输出思考内容（该行为由 `thinking.type` 控制）。推荐把 `keep: "all"` 与 `type: "enabled"` 搭配使用。

使用方式

使用 `keep: "all"` 时，需要把每一轮历史 assistant 消息中的 `reasoning_content` 原样保留在 `messages` 中。最简单的做法是把上一轮 API 返回的 assistant message 直接 append 回 `messages`。

curl python

KIMI 开放平台

```
-H "Content-Type: application/json" \  
-H "Authorization: Bearer $MOONSHOT_API_KEY" \  
-d '{  
  "model": "kimi-k2.6",  
  "messages": [  
    {"role": "system", "content": "你是 Kimi。"},  
    {"role": "user", "content": "第一个问题..."},  
    {  
      "role": "assistant",  
      "reasoning_content": "<上一轮 API 返回的 reasoning_content>",  
      "content": "<上一轮 API 返回的最终回答>"  
    },  
    {"role": "user", "content": "请基于之前的分析继续推导下一步。"}  
  ],  
  "thinking": {  
    "type": "enabled",  
    "keep": "all"  
  }  
}'
```

 `reasoning_content` 会计入 token 消耗。开启 Preserved Thinking 后，历史思考内容会持续占用上下文长度并计费，请酌情使用。

常见问题

Q1: 为什么需要保留 reasoning_content?

A: 保留 reasoning_content 可以确保模型在多步推理过程中保持推理的连贯性，特别是在工具调用过程中。服务器会自动处理这些字段，用户无需手动管理。

Q2: reasoning_content 会消耗额外的 token 吗?

A: 是的，reasoning_content 会计入输入/输出 token 消耗。具体计费方式请参考 MoonshotAI 的定价文档。

 Kimi K2.6 模型已正式发布，长程代码编写能力更强更稳。

KIMI 开放平台



< Kimi K2.6 多模态模型

多轮对话指南 >

>

